DA3408 – Assignment 5

SwiftParcel ETA Service – Report

Kiran Kumar P
DA24B008

1 Introduction

This assignment builds and releases an ETA (estimated delivery time) model as a production service: a validated FastAPI REST service with metrics and a load test (Part A), an MLflow registry with a promotion gate and a shadow comparison (Part B), a sticky canary on Ray Serve (Part C) and a release smoke test with canary abort criteria (Part D). All numbers come from runs on my laptop on 5 Nov 2026; the raw outputs are in `partA/*.json`, `partB/gate_output.txt`, `partB/shadow_output.txt` and `partC/verify_output.txt`, with screenshots of every run in `screenshots/`.

Machine. AMD Ryzen 7 8840HS (8 cores / 16 threads), 14 GiB RAM, Ubuntu (Linux 7.0), Python 3.10.22 (conda). fastapi 0.162.0, uvicorn 0.54.0, pydantic 2.14.3, scikit-learn 1.7.2, mlflow 3.18.0, ray 2.60.0, prometheus_client 0.26.0. The load generator ran on the same laptop as the server. `python eta_data.py`: v1 (Ridge) fresh MAE 11.95 min, v2 (GBR) fresh MAE 6.10 min.

2 Part A: REST Service

2.1 A1. Service (`partA/eta_app.py`)

- `POST /v1/models/eta:predict` with a pydantic `PredictRequest`; predictions rounded to 1 decimal, in input order.
- `Field(ge/le/gt)` on every feature; `instances` has `min_length=1, max_length=500`.
- Cross-field rule (`model_validator`): `distance_km > 150` and `n_stops > 20` ⇒ "infeasible" ⇒ 422.
- Middleware takes `X-Request-ID` or generates a `uuid4` and echoes it on every response (including 422 and 500).
- `RequestValidationError` handler returns `{"error": "validation_failed", "details": [{field, msg}], "request_id"}`, with `field` like `instances.0.hour`.
- Own `CollectorRegistry`: counter `eta_requests_total{path, status}` and histogram `eta_request_latency_seconds` (11 buckets + `+Inf`). `/metrics` is skipped, and `path` is the route template, so random URLs cannot create new label values.
- `GET /healthz`; inference is wrapped in `try/except` and returns a generic 500 `{"error": "internal_error", "request_id"}`; the traceback is logged on the server only.
- The bundle is loaded once at import; the endpoint is a plain `def`, so sklearn runs in FastAPI's thread pool (as in Lecture 10 `rest_app.py`).

2.2 A2. Tests (`partA/test_eta_app.py`): 15 passed

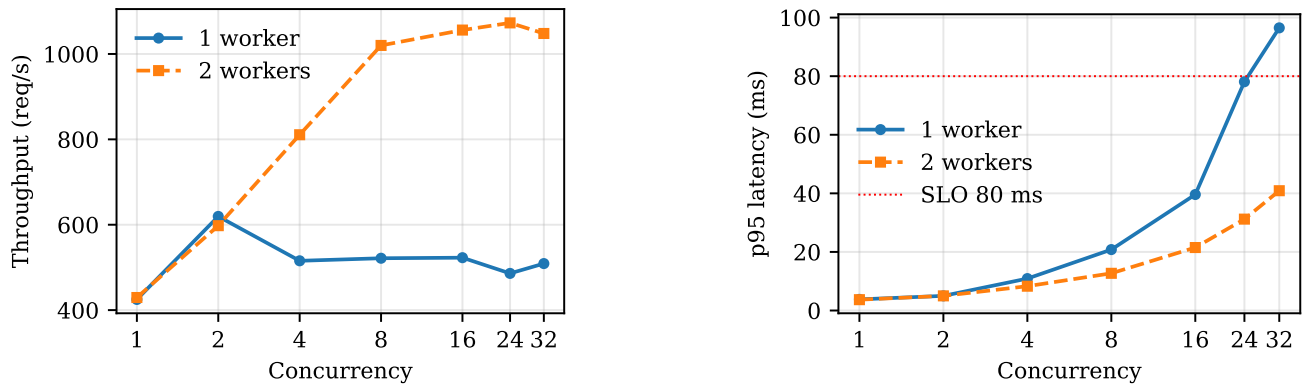
Valid request; batch order; 5 kinds of invalid input (out of range, wrong type, missing field, `rain=2`, `weight_kg=0`); the cross-field rule at both edges (150/21 and 151/20 allowed); batch sizes 0, 500 and 501; request-id echo, generation and on a 422; `/healthz`; the metrics text (counter goes up by exactly 1 for 200 and 422, `+Inf`, `_sum`, `_count`, no `/metrics` series); and a generic 500 from a monkeypatched model whose exception text must not leak. **Mutation check:** changing `150→1500` and removing the header echo in a copy of the app makes **4 tests fail**.

2.3 A3. Load Test (v1, batch 1, 4 s per level)

Concurrency	Requests	rps	p50 ms	p95 ms	p99 ms	Errors	X · p50
1	1700	425.0	2.0	3.8	5.2	0	0.8
2	2478	**619.5**	2.9	5.0	6.3	0	1.8
4	2062	515.5	7.5	10.9	13.9	0	3.9
8	2086	521.5	15.0	20.8	25.7	0	7.8
16	2091	522.8	29.4	39.6	85.3	0	15.4
24 (extra)	1944	486.0	46.4	**78.1**	153.3	0	22.6
32 (extra)	2036	509.0	60.8	**96.5**	129.3	0	30.9

- **Saturation.** Throughput stops growing after concurrency 2 and stays near 520 req/s. Little's Law, $L = X \cdot W$ (p50 as W), returns the concurrency set ($7.8 \approx 8$, $15.4 \approx 16$): once saturated, extra clients only queue, $W \approx N/X_{\max}$, and p50 doubles each time N doubles. `top` at $c = 8$ shows one uvicorn process at ~92% CPU (one core, the GIL) while the machine is 88% idle. Service time $1/X_{\max} \approx 1.6$–1.9 ms. **Saturation point: concurrency ≈ 2, at about 520–620 req/s.**
- **Capacity under p95 ≤ 80 ms.** With $X_{\max} \approx 520$, $W = N/520$, and p95 ≈ 1.35–1.7× p50, so p95 reaches 80 ms at p50 ≈ 47–59 ms, i.e. $N \approx 24$–31. Measured: $c = 24 \to 78.1$ ms (pass), $c = 32 \to 96.5$ ms (fail). **Capacity ≈ 500 req/s with up to ~24 requests in flight**; plan on ~400 req/s (80%) since p99 already exceeds 80 ms at $c = 16$.
- **Batch 1 vs 100 ($c = 1$).** Batch 1: 447 req/s ⇒ **2.24 ms/row**. Batch 100: 313 req/s ⇒ 3.19 ms/request ⇒ **0.032 ms/row** (~70× cheaper). Most of the cost is fixed (HTTP, JSON, pydantic, DataFrame, response); an extra row adds only $(3.19 - 2.24)/99 \approx 0.01$ ms because Ridge is a vectorised matrix product. The price is latency: the first row waits for the last.
- **Why p95/p99, not the mean.** The mean hides the tail users actually see: at $c = 16$, p50 is 29 ms but p99 is 85 ms. An SLO is a per-request promise, and the tail is where queueing shows first.

Change: 2 uvicorn workers. The CPU-bound work runs under one GIL per process, so a second process should double capacity.



(a) Throughput

(b) p95 latency vs. SLO

Figure 1: Load test, 1 vs. 2 uvicorn workers (from `partA/loadtest_*.json`).

- At the same concurrency: **1.96× ($c = 8$) and 2.02× ($c = 16$)**; peak vs. peak 1072.8/619.5 = **1.73×**. At $c = 1$–2 nothing changes, since one request at a time cannot use two processes.
- p95 at $c = 32$ drops from 96.5 to 40.9 ms, so capacity under the SLO is now ≈ 1,050 req/s. `top` shows both workers and the load generator at ~100% CPU, so the generator is now also a limit.
- Side effect: `prometheus_client` counters are per process, so `/metrics` shows one worker's counts (needs `PROMETHEUS_MULTIPROC_DIR`; not done).

3 Part B: MLflow Registry, Gate and Shadow

3.1 B1. Registration (`partB/eta_mlflow.py register`)

`EtaPyfunc` wraps the bundle and returns `eta_minutes` (1 decimal) and `model_version`. Each version is logged with `infer_signature` (7 float64 inputs, 2 outputs), a 2-row `input_example`, pinned `pip_requirements` (scikit-learn 1.7.2, pandas 2.3.3, numpy 2.2.6, joblib 1.6.0, mlflow 3.18.0), params `algorithm`, `bundle_version` and metric `fresh_mae`. Tracking store: `sqlite:///mlflow.db`.

Version	Bundle	Algorithm	Alias
1	v1	Ridge (scaler + Ridge pipeline)	**prod**
2	v2	GradientBoostingRegressor	**candidate**

3.2 B2. Promotion Gate

- `decide()` is pure (no I/O, inputs unmodified) and returns 4 booleans plus the numbers: `mae_gain`: (prod − cand)/prod ≥ `min_rel_gain`; `latency_p95`: cand p95 ≤ `max_p95_ms`; `rush_slice`, `rain_slice`: cand slice MAE ≤ prod slice MAE ×(1+`max_slice_regression`). "Not more than 0.05" is read as relative (5%), like the gain; a 10^{-9} tolerance stops float error flipping values exactly on a threshold.
- `promotion_gate()` loads `@prod` and `@candidate`, evaluates both on `make_data(2000, seed=99)` (unseen; training used seed 1) and times 200 single-row predictions after 10 warm-ups.
- `test_gate.py`: **13 passed** (on, just inside and just outside every threshold; a candidate that wins overall but regresses on rain; custom thresholds; purity).

Alias	MAE	MAE rush	MAE rain	p95 ms (in-process)
prod (v1)	11.955	19.680	11.623	0.90
candidate (v2)	6.105	8.580	6.285	1.05

Default thresholds: relative gain 0.489 ≥ 0.03, all 4 criteria true ⇒ **promote = true**. With `min_rel_gain=0.60`: `mae_gain` false, the rest true ⇒ **promote = false**.

3.3 B3. Shadow Comparison

200 rows of `eta_fresh.csv`, one row per request, the same request sent to both servers.

Metric	Value
Mean |prod − candidate|	**9.48 min** (max 51.7)
Requests with |diff| > 10 min	**35.0%**
p95 latency, prod :5101 / candidate :5102	**6.34 / 6.91 ms** (p50 4.49 / 4.68)

- **Decision.** One ETA in three changes by more than 10 minutes. That is expected: v2 halves the fresh error, mostly in rush hour (19.7 → 8.6 min), where Ridge cannot fit the rush × traffic interaction. Latency is almost the same and far below the SLO, and the gate passes on every criterion, so **promote v2** – but through the sticky canary (10% → 50% → 100% of customers) with the Part D abort criteria, checking MAE against real delivery times as they arrive.
- **Moving an alias does not change a running server.** `mlflow models serve -m models:/eta-model@prod` resolves the alias once at start-up, downloads that version and keeps it in memory; it never asks the registry again. Verified: with `prod` moved to version 2, the running :5101 server still answered `model_version: v1`.
- **Picking up the new version without dropping requests.** (1) *Rolling / blue-green restart*: start new processes (pods) that resolve the alias again, wait for readiness and the smoke test, switch the load balancer or Kubernetes Service, then drain the old ones (`RollingUpdate maxUnavailable: 0`, Lecture 10 Minikube). (2) *Hot swap*: a background thread polls the alias (or gets a webhook), loads and warms the new version beside the old one, then swaps the model reference atomically; in-flight requests finish on the old model.

4 Part C: Ray Serve Sticky Canary

4.1 C1. Application (`partC/eta_ray.py`)

- `EtaModelV1` (prod): `num_replicas=2`. `EtaModelV2` (candidate): autoscaling min 1 / max 3, `target_ongoing_requests=2`, upscale 3 s, downscale 30 s, metrics interval 2 s, look-back 10 s. A FastAPI ingress

`Router` takes `{"customer_id", "instances"}` and returns the model output plus `"arm"`.

- **Sticky split:** bucket(id) = first 8 bytes of sha256(id)/2^{64}; candidate iff bucket $<$ weight. Routing is per customer, and because the condition is monotonic, raising the weight only moves customers from prod to candidate. `X-Beta: 1` forces v2.
- CPUs: router 0.1 + prod 2×0.25 + candidate $\leq 3 \times 0.25 =$ **1.35 of 2 CPUs** (`ray status`: 0.85/2.0 at minimum scale).

4.2 C2. Verification (`partC/verify_ray.py`, `num_cpus=2`, 1000 ids, weight 0.2): all 5 checks pass

Check	Evidence
(a) Share	208/1000 = 0.208 on v2 (allowed 0.16–0.24)
(b) Second call	200/200 same arm
(c) After `serve.delete` + redeploy	200/200 same arm
(d) Weight 0.2 → 0.5	candidate → prod: **0**; prod → candidate: 312; new share 0.520
(e) `X-Beta: 1`	50/50 on v2 (same customers without the header: v1)

4.3 C3. Short Answers

1. **Why not `hash()`?** For `str`, Python salts the hash per process (`PYTHONHASHSEED`), so every router replica and restart buckets customers differently – breaking checks (b)–(d). It is also not stable across Python versions or platforms. sha256 is deterministic everywhere and spreads ids evenly.
2. **Why stickiness matters.** *Product:* a customer refreshing the app must not see the ETA jump between models (41 → 55 → 41 min); flip-flopping ETAs lose trust. *Analysis:* the customer is the unit of randomisation; if one customer saw both arms, outcomes (late deliveries, complaints) could not be attributed to either model, and observations would not be independent. Stickiness lets delayed ground truth join back to one arm.
3. **Autoscaling parameters.** A request takes ∼2–5 ms, so `target_ongoing_requests=2` scales up as soon as a queue forms without a replica per request. Scale up fast (3 s) because canary traffic is small and bursty; scale down slowly (30 s) to avoid flapping. A 10 s look-back with 2 s metrics smooths single spikes. `max_replicas=3` comes from the CPU budget, and `min_replicas=1` keeps a warm replica so the first canary customer avoids a cold start.

5 Part D: Release Gate

`partD/smoke_eta.py --url … --expect-version … [--max-p95-ms 80]` runs 8 checks and exits 0 or 1: `/healthz`; expected `model_version`; a 20-row batch (200, finite, 0–2000 min); **3 model-sanity checks** (ETA does not decrease as distance goes 5 → 40 → 120 km, traffic 0.1 → 0.5 → 0.9, or stops 1 → 8 → 20); **invalid input (`hour=25`) → 422**; and p95 of 30 single-row requests ≤ limit. It works against both Part A and Part C. Runs: Part A v1 PASS (p95 2.5 ms), Part A v2 PASS (4.8 ms), Ray app v1 PASS (20.1 ms); **exit 1** for a wrong expected version and for an impossible `--max-p95-ms 0.5`.

Abort criteria for a 10% canary of v2 (v2 arm vs. v1 over the same window; any one ⇒ set weight to 0):

#	Metric	Abort if	Window
1	5xx ratio of the v2 arm (`eta_requests_total{status=~"5.."}` / all)	$> 1\%$ **or** $> 2\times$ v1's ratio	5 min rolling, ≥ 200 v2 requests
2	v2 p95 latency (`histogram_quantile(0.95, …)`)	> 80 ms **or** $> 1.5\times$ v1 p95	3 consecutive 1-min windows
3	MAE vs. real delivery times, v2 vs. v1 (overall, rush, rain)	v2 overall MAE $>$ v1's, or any slice $>$ v1 + 5%	rolling 1 h, ≥ 300 completed v2 deliveries
4	Late-delivery rate (delivered > 15 min after the ETA shown)	$>$ v1's rate + 2 pp	rolling 1 h, ≥ 300 completed v2 deliveries

6 Not Tested

- Autoscaling of the candidate under real load: only the config and the idle state (1 replica) were checked; no load test scaled it to 3.

- The abort criteria are a design only: no Prometheus server scraped `/metrics` (the text format was checked by tests and curl), and no alerting was set up.
- Client and server shared one laptop; with 2 workers the load generator was CPU-bound, so the 2-worker numbers are a lower bound.
- `/metrics` with `--workers 2` is per process (multiprocess mode not set up).
- `smoke_eta.py` was not run against the MLflow servers (different API: `/invocations`, 400 with no range checks).
- The shadow test uses 200 offline rows, not mirrored live traffic; the gate's p95 is in-process `pyfunc.predict`, not over HTTP.
- Only Python 3.10 on Linux; no Docker / Kubernetes deployment (not asked).

7 Sources and Reuse

The starter `eta_data.py` and `load_test.py` are used unchanged. Ideas and small fragments reused, with changes: Lecture 10 *REST_API_Serving_Exercise* (`rest_app.py`: bundle loaded once, pydantic schema, timing middleware, thread-pool `def` endpoint, the `--workers` experiment); *MLflow_Serving_Exercise* (`mlflow_register.py`: pyfunc wrapper, float64 signature, `log_model` + alias; the gate on fresh data and the shadow loop from its Tasks A/B); *Ray_Serve_Exercise* (`ray_serve_app.py`: model deployment + FastAPI Router + `build_app`, fractional CPUs, `extra_work_ms`, autoscaling keys); Lecture 10 *Minikube_Serving* slides (smoke-test checks and `smoke || rollback`, rolling update for B3); Lecture 9 and my Assignment 4 (Prometheus counter and histogram).